

Operator: ReLU

Phase 1.5 nấc 2 → đầu vào cho Phase 2A (viết operator) và Phase 2D (golden model C).

1. Vai trò

Cắt bỏ phần âm: số dương giữ nguyên, số âm ép về 0.

Đây là nguồn phi tuyến **được đặt vào có chủ đích** của `SmallCNN` — nhưng **không phải nguồn duy nhất**.

`Conv2d`, `Flatten` và `Linear` đều tuyến tính, nên nếu mạng chỉ có ba phép đó thì xếp bao nhiêu lớp cũng vô ích: tích của nhiều phép tuyến tính vẫn là **một** phép tuyến tính. Đó là lý do ReLU tồn tại.

Chỗ dễ nói hớ: `MaxPool2d` **cũng phi tuyến** — $\max(a, b)$ không phải phép tuyến tính. Nên bỏ hết ReLU đi thì mạng **không** sụp về một lớp tuyến tính: ba lớp pool vẫn còn đó, mạng vẫn là hàm phi tuyến (tuyến tính từng khúc). Nó chỉ yếu đi rất nhiều, vì phi tuyến của $\max$ chỉ là chọn lớn nhất trong cửa sổ, không học được gì và không đổi dấu được. Xem đáp án Câu 1 ở mục 7.

2. Công thức

$$y = \max(0, x)$$

Áp dụng **từng phần tử một** (element-wise). Shape vào = shape ra, không đổi chiều nào, không có tham số nào.

Đạo hàm (chỉ dùng khi train, inference không cần):

$$\begin{aligned} dy/dx &= 1 && \text{nếu } x > 0 \\ &= 0 && \text{nếu } x \leq 0 \end{aligned}$$

Đạo hàm bằng đúng 1 ở nhánh dương là lý do ReLU không bị vanishing gradient như `sigmoid` hay `tanh` — gradient đi qua bao nhiêu lớp vẫn giữ nguyên độ lớn.

3. Mã giả

```
relu(data[N]):                                # N = tổng số phần tử, không quan tâm shape
    for i = 0 .. N-1:
        if data[i] < 0:
            data[i] = 0
```

Ba dòng. Không nhân, không chia, không cấp phát thêm bộ nhớ — sửa tại chỗ được.

4. Áp dụng vào `SmallCNN`

Vị trí	Shape khi đó	Số phần tử phải xét
sau <code>conv1</code>	$8 \times 28 \times 28$	6.272
sau <code>conv2</code>	$16 \times 14 \times 14$	3.136
sau <code>conv3</code>	$16 \times 7 \times 7$	784
	Tổng	10.192

Đặt cạnh tổng MAC của 3 lớp conv (**395.136**), ReLU chiếm khoảng **2,6%** số phép tính và **0** tham số. Rẻ tới mức không cần bàn khi tối ưu.

5. Mức FPGA-friendly

✔ **Tốt nhất trong mọi operator của mạng này.** Không nhân, không chia, không DSP slice. Chỉ là: đọc bit dấu, nếu là 1 thì xuất 0, ngược lại xuất nguyên. Một comparator + một mux, khoảng 1 LUT cho mỗi phần tử.

Với int8 đối xứng (`zero_point = 0`), ReLU rút gọn xuống mức **một cổng logic**: lấy bit dấu (MSB) đảo lại rồi AND với cả byte.

Một tối ưu bỏ túi: đổi thứ tự `relu` và `pool`

`relu(maxpool(x))` cho kết quả **y hệt** `maxpool(relu(x))`, vì cả `max` lẫn `relu` đều là hàm đơn điệu không giảm:

- Nếu cả cửa sổ đều âm: vế trái = `relu(số âm)` = 0; vế phải = `max(0, 0, 0, 0)` = 0.
- Nếu có ít nhất một số dương: cả hai vế đều trả về đúng số dương lớn nhất.

Mà `maxpool` giảm số phần tử, nên làm ReLU **sau** pool thì rẻ hơn:

Thứ tự	conv1	conv2	conv3	Tổng phép ReLU
<code>pool(relu(x))</code> — code hiện tại	$8 \cdot 28 \cdot 28 = 6.272$	$16 \cdot 14 \cdot 14 = 3.136$	$16 \cdot 7 \cdot 7 = 784$	10.192
<code>relu(pool(x))</code>	$8 \cdot 14 \cdot 14 = 1.568$	$16 \cdot 7 \cdot 7 = 784$	$16 \cdot 3 \cdot 3 = 144$	2.496

Giảm **4,08 lần**, kết quả không đổi một bit.

Không phải đúng 4 lần — đây là chỗ dễ ghi sai. Lấy $10.192 \div 4 = 2.548$ là sai 52 phép. Hai pool đầu chia đúng 4 ($28 \rightarrow 14$, $14 \rightarrow 7$), nhưng pool cuối $7 \rightarrow 3$ bị `floor` cắt hàng và cột cuối: $16 \cdot 7 \cdot 7 = 784$ phần tử vào mà chỉ ra $16 \cdot 3 \cdot 3 = 144$, tức chia 5,44 chứ không phải 4. Đúng cái hành vi `floor` đã nói ở `conv2d.md` — nó không chỉ đổi shape, nó đổi cả phép đếm. Đây đúng loại "thay/bỏ

phép toán cho FPGA" mà project yêu cầu. **Lưu ý:** chỉ đúng với `MaxPool`. Với `AvgPool` thì sai, vì trung bình của các số đã cắt âm khác trung bình rồi mới cắt.

6. Chú ý khi port sang C

Bẫy	Chi tiết
Sửa tại chỗ	<code>nn.ReLU(inplace=True)</code> ghi đè mảng đầu vào. Trong C làm vậy cũng được với inference, nhưng nếu còn cần giá trị gốc ở đâu đó thì mất
<code>** <</code> hay <code><= **</code>	<code>max(0, x)</code> với <code>x = 0</code> ra 0 dù viết kiểu nào. Không có bẫy ở đây — khác với <code>argmax</code>
Dead ReLU	Nếu một kênh cho ra ≤ 0 với mọi ảnh thì kênh đó chết, trọng số conv sinh ra nó là vô dụng. Đáng kiểm khi muốn cắt tiếp params
Số âm trong fixed-point	Q1.7 dùng bù 2. Kiểm tra bit dấu, đừng so sánh như số không dấu

7. Tự kiểm tra

Làm trên giấy:

- Bỏ toàn bộ ReLU khỏi `SmallCNN`. Mạng còn lại tương đương với **một** lớp gì? Trong 5.018 tham số, bao nhiêu trở thành thừa?
- Áp dụng tối ưu ở mục 5 cho riêng vị trí sau `conv1`: số phép ReLU ở đó giảm từ bao nhiêu xuống bao nhiêu?
- Với int8 `zero_point = 0`, viết ReLU bằng **một** biểu thức bit (không dùng `if`).

Đáp án

Câu 1 — Bỏ toàn bộ ReLU thì còn lại gì

Câu trả lời ngắn: mạng KHÔNG sụp về một lớp tuyến tính. Đây là chỗ bẫy.

Ba phép `Conv2d`, `Flatten`, `Linear` đều tuyến tính, nên nếu mạng chỉ có ba thứ đó thì đúng là chồng bao nhiêu lớp cũng chỉ tương đương **một** phép biến đổi tuyến tính. Nhưng `SmallCNN` còn ****3** lớp `MaxPool2d`, mà `max(a, b)` **không tuyến tính**:

$$\begin{aligned} \max(1, 3) + \max(2, 0) &= 3 + 2 = 5 \\ \max(1+2, 3+0) &= \max(3, 3) = 3 \quad \rightarrow \quad 5 \neq 3 \end{aligned}$$

Nên bỏ hết ReLU thì mạng vẫn là hàm **phi tuyến, tuyến tính từng khúc** — cụ thể là hợp của các phép affine với phép lấy max, dạng hàm "max của nhiều hàm affine".

Vậy mất gì khi bỏ ReLU? Phi tuyến của `max` yếu hơn hẳn phi tuyến của `relu` ở hai điểm:

	ReLU	MaxPool
Có tham số học được không	Không, nhưng đặt sau mỗi conv nên quyết định kênh nào được đi tiếp	Không
Xử lý số âm	Cắt hẳn về 0 — tạo ra vùng phẳng, đây mới là thứ cho mạng "tắt" đặc trưng	Giữ nguyên số âm nếu cả cửa sổ đều âm
Mật độ	Trên từng phần tử , 10.192 vị trí	Chỉ 1 lần mỗi cửa sổ 2x2

Bao nhiêu tham số thành thừa: không tham số nào thành thừa theo nghĩa toán học — cả 5.018 vẫn tham gia tính toán và vẫn ảnh hưởng đầu ra. Nhưng **khả năng biểu diễn tụt mạnh**, vì mạng mất toàn bộ khả năng cắt âm. Muốn biết tụt bao nhiêu thì phải train thử, không suy ra trên giấy được.

Nói với Thầy thì gọn thế này: ReLU là phi tuyến được đặt vào có chủ đích, nhưng MaxPool cũng phi tuyến — bỏ ReLU không làm mạng thành tuyến tính, chỉ làm nó yếu đi nhiều.

Câu 2 — Áp tối ưu riêng cho vị trí sau conv1

```
hiện tại pool(relu(x)) : 8 × 28 × 28 = 6.272 phép ReLU
tối ưu   relu(pool(x)) : 8 × 14 × 14 = 1.568 phép ReLU
```

Giảm **đúng 4 lần** — khác với tổng toàn mạng (4,08 lần). Vì 28 → 14 chia hết cho 2, không bị floor cắt hụt như pool3 (7 → 3, chia 5,44).

Câu 3 — Biểu thức bit cho int8, zero_point = 0

```
y = x & ~(x >> 7); // >> là dịch phải CÓ DẤU (arithmetic shift)
```

x	x >> 7	~(x >> 7)	x & ...
âm	0xFF (toàn bit 1)	0x00	0
≥ 0	0x00	0xFF	x

Một phép dịch + một phép NOT + một phép AND, không rẽ nhánh. Trên FPGA còn rẻ hơn nữa: chỉ cần lấy bit dấu (MSB) đảo lại rồi AND với cả byte — khoảng 1 LUT cho mỗi phần tử.

Bẫy: phải là dịch phải **có dấu**. Trong C, >> trên kiểu unsigned là dịch logic (chèn bit 0), biểu thức này sẽ sai với số âm. Khai báo int8_t, đừng uint8_t.